



VOLUME 05 / RUST BACKEND MASTERY

Rustバックエンド — async・並行処理・ジョブ・運用

Futureの仕組みから過負荷、キャンセル、ログ、ジョブ運用まで、サーバーとしてRustを使うための中核を学ぶ

10

深掘り単元

20+

実行・解答例

40+

図・表・画面

実務

設計と障害対応

LEARNING MAP

この巻で身に付けること

async構文だけでなく、taskの所有者、backpressure、timeout、blocking隔離、再試行、graceful shutdownを障害時の挙動から設計します。

到達目標

- ✓ Futureとruntimeの役割を説明できる
- ✓ task・channel・Semaphoreで負荷を制御できる
- ✓ timeout・cancel・shutdownを実装できる
- ✓ ログ・再試行・jobを運用可能な形に設計できる

学習地図

01 async・Future・awaitの実体

async fnは呼び出しただけでは処理を進めず、Futureという状態機械を返します。executorがpollし、I/O待ちでPendingなら別タスクを進め、準備できたら再開します。

02 Tokio runtime・scheduler・spawn

Tokio runtimeはFutureを実行するscheduler、I/O driver、timerを提供します。tokio::spawnは独立taskを作り、通常はStatic + Sendを要求するため、借用値をそのまま持ち込めません。

03 JoinSet・select!・構造化された並行処理

複数処理を同時に進めるとき、親処理が子taskの寿命と失敗を管理します。select!は最初に完了した枝を選び、残るfutureのキャンセル安全性を考慮します。

04 channel・backpressure・所有権移動

mpsc channelはproducerからconsumerへ値の所有権を移します。bounded channelの容量が満杯なら送信側を待たせ、処理能力を超えた入力へbackpressureを掛けます。

05 Mutex・RwLock・Semaphoreとawait

同期原語は共有状態または同時実行数を守ります。std::sync::MutexGuardをawait越しに保持するとexecutorを妨げるため、必要データを取り出してguardをdropしてからawaitします。

06 timeout・cancellation・graceful shutdown

外部I/Oには必ず時間上限を設け、サーバー停止時は新規受付を止め、処理中リクエストとworkerへ猶予を与えてから終了します。キャンセル後も不変条件が守られる単位にします。

07 blocking処理・spawn_blocking・CPU負荷

標準ファイルI/O、Argon2、圧縮など長い同期・CPU処理をruntime worker上で行うと他taskが進みません。spawn_blockingへ隔離し、同時数と入力サイズを制限します。

08 HTTP client・retry・idempotency

外部API呼び出しはDNS、接続、TLS、応答、JSON解析で失敗します。全エラーを再試行せず、timeoutや一時的5xxだけを指数backoffとjitterで再試行し、操作の冪等性を確認します。

09 tracing・構造化ログ・メトリクス

printlnは文字列だけですが、tracingはspanとfieldでrequest_id、user_id、operation、latencyを構造化します。ログlevelと機密除外を決め、メトリクス・traceと関連できるようにします。

10 background job・設定・運用境界

バックグラウンド処理には投入、予約、取得、lease、成功、再試行、dead-letterの状態があります。メモリtaskでは再起動に弱いため、重要処理はDBへ永続化し複数workerでも一度だけ実行します。

HOW TO USE THIS VOLUME

読むだけで終わらせない進め方

「予想→実行→観察→一か所変更→回帰テスト」を一单元ごとに繰り返します。写経した行数ではなく、入力・処理・出力・失敗を自分の言葉で説明できるかを進捗にします。

1. 予想

実行前に入力型、出力、失敗箇所を予想します。

2. そのまま実行

最初は変更せず、教材と同じ出力が比較します。

3. 一か所変更

値または型を一つだけ変え、差分の理由を説明します。

4. 回帰テスト

直した失敗を自動テストへ残し、再発を防ぎます。

STEP 1

Futureは遅延計算

STEP 2

pollとWaker

STEP 3

awaitで中断

STEP 4

I/O並行性

毎回そろえる確認コマンド

● ● ● 単元を終える前の品質確認

```
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test --locked --all-targets
```

対象	この教材の基準	混在を防ぐ理由
Rust	2024 Edition / rust-version 1.95以上	editionとMSRVを明示し、環境差を再現可能にする
Topcoat	crate / CLI ともに 0.5.0固定	early-stageのため、mainブランチの別構文を混ぜない
DB・認証	Turso/libSQL、Argon2id、Topcoat Session	一つのRustプロセスで責務とデータの流れを追える
公開	Docker + Fly.io、HOST=0.0.0.0、PORT=8080	ローカルとコンテナの待受差を明示する

重要な版の前提 Topcoatは公式がearly-stage / experimentalと明記しています。本教材はcrateとCLIを **0.5.0** に固定し、mainブランチの別構文を混ぜません。

UNIT 01

async・Future・awaitの実体

UNIT 01・CONCEPT

async・Future・awaitの実体

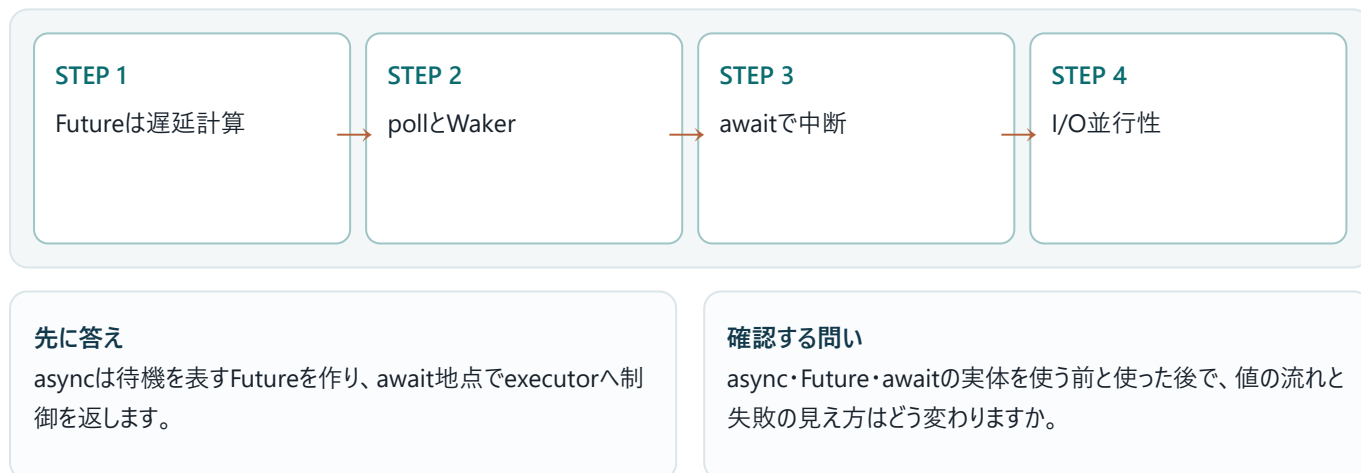
async fnは呼び出しただけでは処理を進めず、Futureという状態機械を返します。executorがpollし、I/O待ちでPendingなら別タスクを進め、準備できたら再開します。

この単元の到達点

- Futureは遅延計算
- pollとWaker
- awaitで中断
- I/O並行性

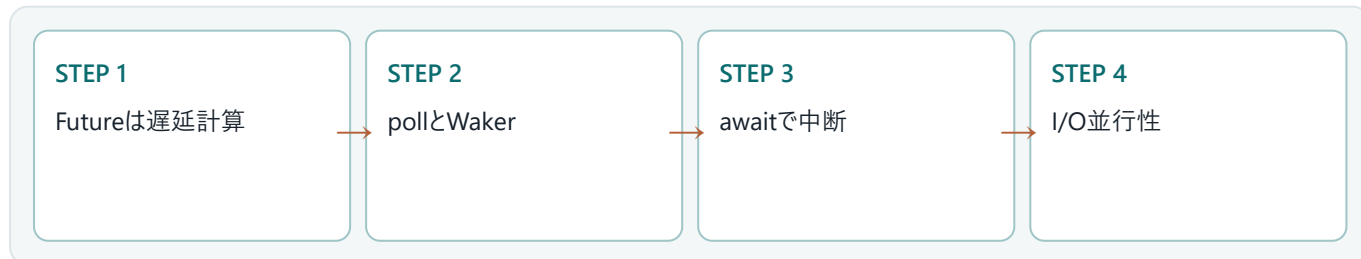
なぜバックエンドで必要か

DB、HTTP、タイマーの待機をasyncで表し、少数スレッドで多数接続を扱います。CPU重処理が速くなる仕組みではありません。



UNIT 01・MENTAL MODEL

async・Future・awaitの実体を図でつかむ



async fnは呼び出しただけでは処理を進めず、Futureという状態機械を返します。executorがpollし、I/O待ちでPendingなら別タスクを進め、準備できたら再開します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	Futureは遅延計算	Futureは遅延計算 — 入力と前提を確認し、境界を曖昧にしない。
2	pollとWaker	pollとWaker — 値がどの順で変換されるかを追跡する。
3	awaitで中断	awaitで中断 — 失敗を再現し、型またはログから原因を特定する。
4	I/O並行性	I/O並行性 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

async・Future・awaitの実体とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 01・MINIMUM CODE

まず動く最小コード

async・Future・awaitの実体だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```

async fn fetch_label(id: u64) -> String {
    tokio::time::sleep(std::time::Duration::from_millis(20)).await;
    format!("note-{}", id)
}

#[tokio::main]
async fn main() {
    let future = fetch_label(7);
    println!("before await");
    println!("{}", future.await);
}

```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 01・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>async fn fetch_label(id: u64) -> String {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>tokio::time::sleep(std::time::Duration::from_millis(20)).await;</code>	非同期処理の完了を待ちます。待機中にスレッドを占有しない設計が確認します。
<code>format!("note-{}", id)</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>#[tokio::main]</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力Futureは遅延計算の条件を満たすか確認する。
- ✓ pollとWakerを実行し、途中の型と状態を追う。
- ✓ awaitで中断で失敗を成功経路から分離する。
- ✓ I/O並行性により、出力と副作用を検証する。

型を声に出す async・Future・awaitの実体に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 01・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
before await
note-7
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 01・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
use std::time::{Duration, Instant};
async fn work(ms:u64){tokio::time::sleep(Duration::from_millis(ms)).await;}
#[tokio::main]async fn main(){let start=Instant::now();tokio::join!(work(50),work(50));println!("elapsed={:?}",start.elapsed());}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

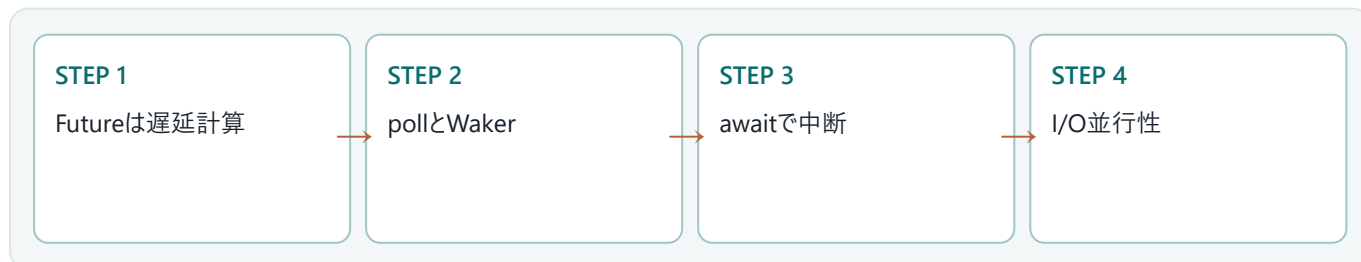
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 01・BACKEND APPLICATION

バックエンドのどこで使うか

DB、HTTP、タイマーの待機をasyncで表し、少数スレッドで多数接続を扱います。CPU重処理が速くなる仕組みではありません。



リクエスト側

- Futureは遅延計算を入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- pollとWakerを小さな責務へ分割
- awaitで中断をResultとログで表現
- I/O並行性をテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 01・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	Futureは遅延計算に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違ふ	pollとWakerの入力または実行順序が想定と違ふ	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	awaitで中断またはI/O並行性の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 01・DEBUGGING

async・Future・awaitの実体を再現して切り分ける

async・Future・awaitの実体の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: Futureは遅延計算
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: pollとWaker
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: awaitで中断
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: I/O並行性

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	Futureは遅延計算	Futureは遅延計算 — 入力と前提を確認し、境界を曖昧にしない。
2	pollとWaker	pollとWaker — 値がどの順で変換されるかを追跡する。
3	awaitで中断	awaitで中断 — 失敗を再現し、型またはログから原因を特定する。
4	I/O並行性	I/O並行性 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 asyncは待機を表すFutureを作り、await地点でexecutorへ制御を返します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 01 · PRACTICE

手を動かす演習

二つのsleep処理を順次awaitした場合とtokio::join!した場合の所要時間を比較してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 01・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
use std::time::{Duration, Instant};
async fn work(ms:u64){tokio::time::sleep(Duration::from_millis(ms)).await;}
#[tokio::main]async fn main(){let start=Instant::now();tokio::join!(work(50),work(50));println!("elapsed=
{:?} ",start.elapsed());}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ asyncは待機を表すFutureを作り、await地点でexecutorへ制御を返します。
- ✓ async・Future・awaitの実体とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 02

Tokio runtime・scheduler・spawn

UNIT 02 ・ CONCEPT

Tokio runtime・scheduler・spawn

Tokio runtimeはFutureを実行するscheduler、I/O driver、timerを提供します。tokio::spawnは独立taskを作り、通常は'Static + Sendを要求するため、借用値をそのまま持ち込めません。

この単元の到達点

- runtime構築
- 協調スケジューリング
- spawnとJoinHandle
- 'static + Send

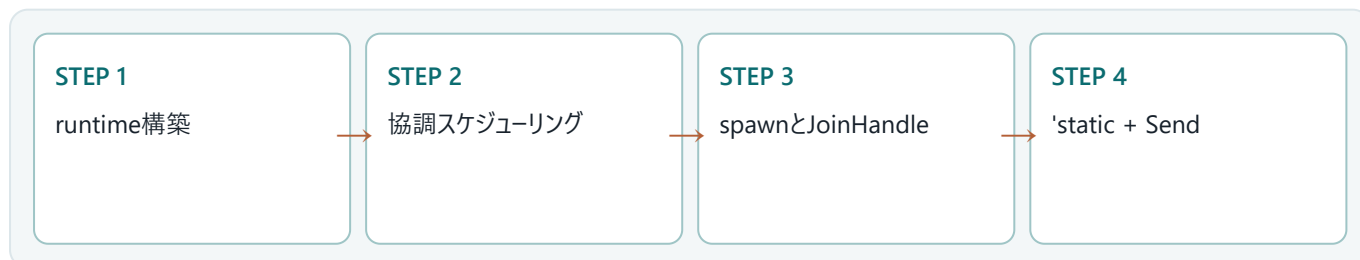
なぜバックエンドで必要か

リクエスト処理から無制限にspawnすると障害時にタスクが増え続けます。所有者、終了条件、同時数、失敗回収を設計します。



UNIT 02 ・ MENTAL MODEL

Tokio runtime・scheduler・spawnを図でつかむ



Tokio runtimeはFutureを実行するscheduler、I/O driver、timerを提供します。tokio::spawnは独立taskを作り、通常は'Static + Sendを要求するため、借用値をそのまま持ち込めません。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	runtime構築	runtime構築 — 入力と前提を確認し、境界を曖昧にしない。
2	協調スケジューリング	協調スケジューリング — 値がどの順で変換されるかを追跡する。
3	spawnとJoinHandle	spawnとJoinHandle — 失敗を再現し、型またはログから原因を特定する。
4	'static + Send	'static + Send — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Tokio runtime・scheduler・spawnとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 02 ・ MINIMUM CODE

まず動く最小コード

Tokio runtime・scheduler・spawnだけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
#[tokio::main]
async fn main() {
    let mut handles = Vec::new();
    for id in 1..=3 {
        handles.push(tokio::spawn(async move { format!("task-{}", id) }));
    }
    for handle in handles { println!("{}", handle.await.unwrap()); }
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 02 ・ LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[tokio::main]</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>async fn main() {</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let mut handles = Vec::new();</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>for id in 1..=3 {</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>handles.push(tokio::spawn(async move { format!("{}", id) }));</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力 runtime 構築の条件を満たすか確認する。
- ✓ 協調スケジューリングを実行し、途中の型と状態を追う。
- ✓ spawn と JoinHandle で失敗を成功経路から分離する。
- ✓ 'static + Send により、出力と副作用を検証する。

型を声に出す Tokio runtime・scheduler・spawn に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 02 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
task-1
task-2
task-3
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 02 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[tokio::main]async fn main(){let mut set=tokio::task::JoinSet::new();for id in 0..5{set.spawn(async move{id*2});}let mut out=vec![];while let Some(r)=set.join_next().await{out.push(r.unwrap());}out.sort();assert_eq!(out,vec![0,2,4,6,8]);}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 02 ・ BACKEND APPLICATION

バックエンドのどこで使うか

リクエスト処理から無制限にspawnすると障害時にタスクが増え続けます。所有者、終了条件、同時数、失敗回収を設計します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 02 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	runtime構築に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	協調スケジューリングの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	spawnとJoinHandleまたは'static + Sendの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 02・DEBUGGING

Tokio runtime・scheduler・spawnを再現して切り分ける

Tokio runtime・scheduler・spawnの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: runtime構築
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 協調スケジューリング
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: spawnとJoinHandle
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 'static + Send

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	runtime構築	runtime構築 — 入力と前提を確認し、境界を曖昧にしない。
2	協調スケジューリング	協調スケジューリング — 値がどの順で変換されるかを追跡する。
3	spawnとJoinHandle	spawnとJoinHandle — 失敗を再現し、型またはログから原因を特定する。
4	'static + Send	'static + Send — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 spawnしたタスクには所有者と結果回収経路を用意します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 02 ・ PRACTICE

手を動かす演習

JoinSetで5タスクを起動し、一つずつ完了結果を回収してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 02・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[tokio::main]async fn main(){let mut set=tokio::task::JoinSet::new();for id in 0..5{set.spawn(async move{id*2});}let mut out=vec![];while let Some(r)=set.join_next().await{out.push(r.unwrap());}out.sort();assert_eq!(out,vec![0,2,4,6,8]);}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ spawnしたタスクには所有者と結果回収経路を用意します。
- ✓ Tokio runtime・scheduler・spawnとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 03

JoinSet・select!・構造化された並行処理

UNIT 03・CONCEPT

JoinSet・select!・構造化された並行処理

複数処理を同時に進めるとき、親処理が子taskの寿命と失敗を管理します。select!は最初に完了した枝を選び、残るfutureのキャンセル安全性を考慮します。

この単元の到達点

- 親子の寿命
- JoinSetで回収
- select!の競争
- キャンセル安全性

なぜバックエンドで必要か

クライアント切断、サーバー停止、タイムアウトのいずれかが先に起きたら処理を止め、DB更新の途中状態をトランザクションで守ります。



UNIT 03・MENTAL MODEL

JoinSet・select!・構造化された並行処理を図でつかむ



複数処理を同時に進めるとき、親処理が子taskの寿命と失敗を管理します。select!は最初に完了した枝を選び、残るfutureのキャンセル安全性を考慮します。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結果の種類と副作用を確認します。

順序	観察対象	初心者が確認すること
1	親子の寿命	親子の寿命 — 入力と前提を確認し、境界を曖昧にしない。
2	JoinSetで回収	JoinSetで回収 — 値がどの順で変換されるかを追跡する。
3	select!の競争	select!の競争 — 失敗を再現し、型またはログから原因を特定する。
4	キャンセル安全性	キャンセル安全性 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

JoinSet・select!・構造化された並行処理とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 03 · MINIMUM CODE

まず動く最小コード

JoinSet・select!・構造化された並行処理だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
use tokio::time::{sleep,Duration};
#[tokio::main]async fn main(){
    tokio::select! {
        _ = sleep(Duration::from_millis(10)) => println!("fast"),
        _ = sleep(Duration::from_millis(100)) => println!("slow"),
    }
}
```

●●● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 03 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>use tokio::time::{sleep,Duration};</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>#[tokio::main]async fn main(){</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>tokio::select! {</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>_ = sleep(Duration::from_millis(10)) => println!("fast"),</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>_ = sleep(Duration::from_millis(100)) => println!("slow"),</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力親子の寿命の条件を満たすか確認する。
- ✓ JoinSetで回収を実行し、途中の型と状態を追う。
- ✓ select!の競争で失敗を成功経路から分離する。
- ✓ キャンセル安全性により、出力と副作用を検証する。

型を声に出す JoinSet・select!・構造化された並行処理に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 03・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

fast

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 03・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
use tokio::time::{sleep,Duration};
#[tokio::main]async fn main(){tokio::select!{_ = tokio::signal::ctrl_c()=>println!("signal"),_ =
sleep(Duration::from_secs(5))=>println!("timeout"),}}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 03・BACKEND APPLICATION

バックエンドのどこで使うか

クライアント切断、サーバー停止、タイムアウトのいずれかが先に起きたら処理を止め、DB更新の途中状態をトランザクションで守ります。



リクエスト側

- 親子の寿命を入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- JoinSetで回収を小さな責務へ分割
- select!の競争をResultとログで表現
- キャンセル安全性をテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 03 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	親子の寿命に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	JoinSetで回収の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	select!の競争またはキャンセル安全性の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 03・DEBUGGING

JoinSet・select!・構造化された並行処理を再現して切り分ける

JoinSet・select!・構造化された並行処理の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 親子の寿命
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: JoinSetで回収
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: select!の競争
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: キャンセル安全性

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	親子の寿命	親子の寿命 — 入力と前提を確認し、境界を曖昧にしない。
2	JoinSetで回収	JoinSetで回収 — 値がどの順で変換されるかを追跡する。
3	select!の競争	select!の競争 — 失敗を再現し、型またはログから原因を特定する。
4	キャンセル安全性	キャンセル安全性 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 並行taskを放置せず、親が完了・失敗・キャンセルを回収します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 03 ・ PRACTICE

手を動かす演習

Ctrl+Cと5秒タイマーをselect!し、どちらが先でも終了理由を表示してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 03 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
use tokio::time::{sleep, Duration};
#[tokio::main]async fn main(){tokio::select!{ _ = tokio::signal::ctrl_c()=>println!("signal"), _ =
sleep(Duration::from_secs(5))=>println!("timeout"),}}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 並行taskを放置せず、親が完了・失敗・キャンセルを回収します。
- ✓ JoinSet・select!・構造化された並行処理とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 04

channel・backpressure・所有権移動

UNIT 04・CONCEPT

channel・backpressure・所有権移動

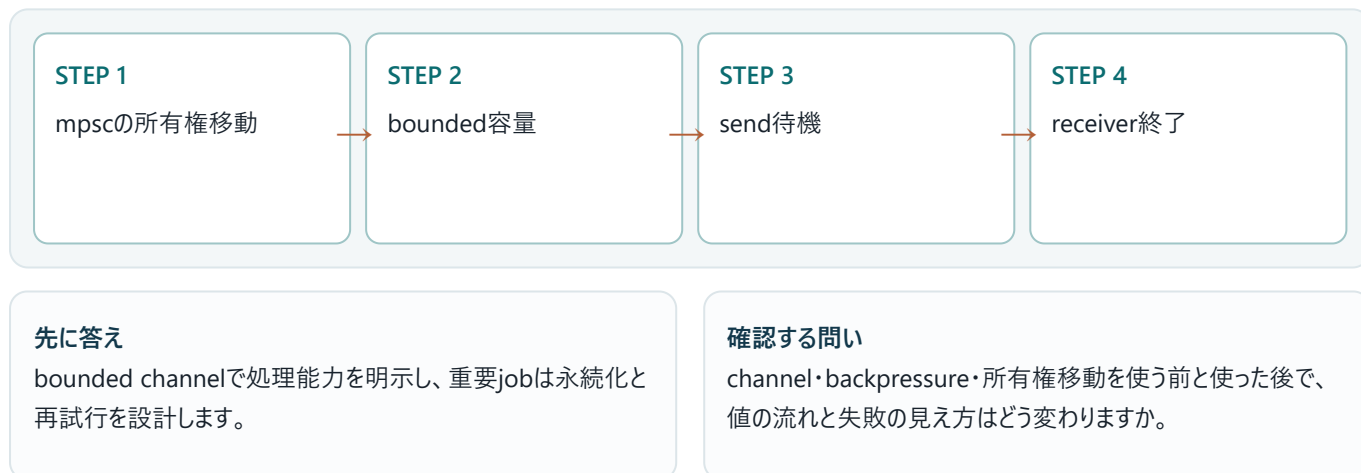
mpsc channelはproducerからconsumerへ値の所有権を移します。bounded channelの容量が満杯なら送信側を待たせ、処理能力を超えた入力へbackpressureを掛けます。

この単元の到達点

- mpscの所有権移動
- bounded容量
- send待機
- receiver終了

なぜバックエンドで必要か

メールや監査イベントをリクエストからworkerへ渡します。メモリchannelだけでは再起動で消えるため、重要jobはDB queueへ永続化します。



UNIT 04・MENTAL MODEL

channel・backpressure・所有権移動を図でつかむ



mpsc channelはproducerからconsumerへ値の所有権を移します。bounded channelの容量が満杯なら送信側を待たせ、処理能力を超えた入力へbackpressureを掛けます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	mpscの所有権移動	mpscの所有権移動 — 入力と前提を確認し、境界を曖昧にしない。
2	bounded容量	bounded容量 — 値がどの順で変換されるかを追跡する。
3	send待機	send待機 — 失敗を再現し、型またはログから原因を特定する。
4	receiver終了	receiver終了 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

channel・backpressure・所有権移動とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 04 · MINIMUM CODE

まず動く最小コード

channel・backpressure・所有権移動だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[tokio::main]async fn main(){
    let (tx, mut rx)=tokio::sync::mpsc::channel:::<String>(2);
    let worker=tokio::spawn(async move{while let Some(job)=rx.recv().await{println!("run {job}");}});
    tx.send("mail-1".into()).await.unwrap();
    tx.send("mail-2".into()).await.unwrap();
    drop(tx); worker.await.unwrap();
}
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 04 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[tokio::main]async fn main(){</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let (tx,mut rx)=tokio::sync::mpsc::channel::<String>(2);</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let worker=tokio::spawn(async move{while let Some(job)=rx.recv().await{pri</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>tx.send("mail-1".into()).await.unwrap();</code>	非同期処理の完了を待ちます。待機中にスレッドを占有しない設計が確認します。
<code>tx.send("mail-2".into()).await.unwrap();</code>	非同期処理の完了を待ちます。待機中にスレッドを占有しない設計が確認します。

実行時に起きる順序

- ✓ 入力がmpscの所有権移動の条件を満たすか確認する。
- ✓ bounded容量を実行し、途中の型と状態を追う。
- ✓ send待機で失敗を成功経路から分離する。
- ✓ receiver終了により、出力と副作用を検証する。

型を声に出す channel・backpressure・所有権移動に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 04 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
run mail-1
run mail-2
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 04 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[tokio::main]async fn main(){let(tx,mut rx)=tokio::sync::mpsc::channel(1);let w=tokio::spawn(async
move{while let
Some(n)=rx.recv().await{tokio::time::sleep(std::time::Duration::from_millis(20)).await;println!("{}",n)};});for n in 0..3{tx.send(n).await.unwrap();}drop(tx);w.await.unwrap();}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 04 · BACKEND APPLICATION

バックエンドのどこで使うか

メールや監査イベントをリクエストからworkerへ渡します。メモリchannelだけでは再起動で消えるため、重要jobはDB queueへ永続化します。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 04 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	mpscの所有権移動に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	bounded容量の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	send待機またはreceiver終了の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 04・DEBUGGING

channel・backpressure・所有権移動を再現して切り分ける

channel・backpressure・所有権移動の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: mpscの所有権移動
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: bounded容量
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: send待機
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: receiver終了

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	mpscの所有権移動	mpscの所有権移動 — 入力と前提を確認し、境界を曖昧にしない。
2	bounded容量	bounded容量 — 値がどの順で変換されるかを追跡する。
3	send待機	send待機 — 失敗を再現し、型またはログから原因を特定する。
4	receiver終了	receiver終了 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 bounded channelで処理能力を明示し、重要jobは永続化と再試行を設計します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 04 · PRACTICE

手を動かす演習

容量1のchannelへ遅いconsumerと三つのjobを流し、sendが待つ時間を計測してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 04 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[tokio::main]async fn main(){let(tx,mut rx)=tokio::sync::mpsc::channel(1);let w=tokio::spawn(async
move{while let
Some(n)=rx.recv().await{tokio::time::sleep(std::time::Duration::from_millis(20)).await;println!("
{n}");}});for n in 0..3{tx.send(n).await.unwrap();}drop(tx);w.await.unwrap();}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ bounded channelで処理能力を明示し、重要jobは永続化と再試行を設計します。
- ✓ channel・backpressure・所有権移動とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 05

Mutex・RwLock・Semaphoreとawait

UNIT 05・CONCEPT

Mutex・RwLock・Semaphoreとawait

同期原語は共有状態または同時実行数を守ります。std::sync::MutexGuardをawait越しに保持するとexecutorを妨げるため、必要データを取り出してguardをdropしてからawaitします。

この単元の到達点

- Mutexの排他
- RwLockの読書分離
- Semaphoreの上限
- guardとawait

なぜバックエンドで必要か

外部APIや重いDB処理の同時数をSemaphoreで制限し、過負荷時の待機上限と拒否方針を決めます。



UNIT 05・MENTAL MODEL

Mutex・RwLock・Semaphoreとawaitを図でつかむ



同期原語は共有状態または同時実行数を守ります。std::sync::MutexGuardをawait越しに保持するとexecutorを妨げるため、必要データを取り出してguardをdropしてからawaitします。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	Mutexの排他	Mutexの排他 — 入力と前提を確認し、境界を曖昧にしない。
2	RwLockの読書分離	RwLockの読書分離 — 値がどの順で変換されるかを追跡する。
3	Semaphoreの上限	Semaphoreの上限 — 失敗を再現し、型またはログから原因を特定する。
4	guardとawait	guardとawait — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Mutex・RwLock・Semaphoreとawaitとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 05・MINIMUM CODE

まず動く最小コード

Mutex・RwLock・Semaphoreとawaitだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
use std::sync::Arc;
use tokio::sync::Semaphore;
#[tokio::main]async fn main(){
    let limit=Arc::new(Semaphore::new(2));
    let mut set=tokio::task::JoinSet::new();
    for id in 0..4{let limit=limit.clone();set.spawn(async move{let
_permit=limit.acquire_owned().await.unwrap();println!("start {id}");});}
    while set.join_next().await.is_some(){}
```

●●● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 05・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>use std::sync::Arc;</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>use tokio::sync::Semaphore;</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>#[tokio::main]async fn main(){</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let limit=Arc::new(Semaphore::new(2));</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let mut set=tokio::task::JoinSet::new();</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。

実行時に起きる順序

- ✓ 入力Mutexの排他の条件を満たすか確認する。
- ✓ RwLockの読書分離を実行し、途中の型と状態を追う。
- ✓ Semaphoreの上限で失敗を成功経路から分離する。
- ✓ guardとawaitにより、出力と副作用を検証する。

型を声に出す Mutex・RwLock・Semaphoreとawaitに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 05・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

同時に最大2つのtaskだけがpermitを保持します。

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 05 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
use std::{sync::Arc, time::Duration}; use tokio::sync::Semaphore;
async fn enter(s: Arc<Semaphore>) -> Result<tokio::sync::OwnedSemaphorePermit, &'static str>
{tokio::time::timeout(Duration::from_millis(100), s.acquire_owned()).await.map_err(|_| "busy")?.map_err(|_| "closed")}
#[tokio::main] async fn main() {assert!(enter(Arc::new(Semaphore::new(1))).await.is_ok());}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 05 ・ BACKEND APPLICATION

バックエンドのどこで使うか

外部APIや重いDB処理の同時数をSemaphoreで制限し、過負荷時の待機上限と拒否方針を決めます。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 05・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	Mutexの排他に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	RwLockの読書分離の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	Semaphoreの上限またはguardとawaitの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 05・DEBUGGING

Mutex・RwLock・Semaphoreとawaitを再現して切り分ける

Mutex・RwLock・Semaphoreとawaitの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: Mutexの排他
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: RwLockの読書分離
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: Semaphoreの上限
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: guardとawait

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	Mutexの排他	Mutexの排他 — 入力と前提を確認し、境界を曖昧にしない。
2	RwLockの読書分離	RwLockの読書分離 — 値がどの順で変換されるかを追跡する。
3	Semaphoreの上限	Semaphoreの上限 — 失敗を再現し、型またはログから原因を特定する。
4	guardとawait	guardとawait — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 ロックは状態、Semaphoreは同時数を守り、awaitをまたぐguardを慎重に扱います。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 05 ・ PRACTICE

手を動かす演習

Semaphore取得を100msでtimeoutし、取得できなければBusyエラーを返してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 05 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
use std::{sync::Arc, time::Duration}; use tokio::sync::Semaphore;
async fn enter(s: Arc<Semaphore>) -> Result<tokio::sync::OwnedSemaphorePermit, &'static str>
{tokio::time::timeout(Duration::from_millis(100), s.acquire_owned()).await.map_err(|_| "busy")?.map_err(|_| "closed")}
#[tokio::main] async fn main() {assert!(enter(Arc::new(Semaphore::new(1))).await.is_ok());}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ ロックは状態、Semaphoreは同時数を守り、awaitをまたぐguardを慎重に扱います。
- ✓ Mutex・RwLock・Semaphoreとawaitとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 06

timeout・cancellation・graceful shutdown

UNIT 06・CONCEPT

timeout・cancellation・graceful shutdown

外部I/Oには必ず時間上限を設け、サーバー停止時は新規受付を止め、処理中リクエストとworkerへ猶予を与えてから終了します。キャンセル後も不変条件が守られる単位にします。

この単元の到達点

- 処理期限
- キャンセル伝播
- 新規受付停止
- 終了猶予

なぜバックエンドで必要か

HTTP client、DB query、queue処理へdeadlineを伝え、途中のDB更新はtransactionで原子的にします。



UNIT 06・MENTAL MODEL

timeout・cancellation・graceful shutdownを図でつかむ



外部I/Oには必ず時間上限を設け、サーバー停止時は新規受付を止め、処理中リクエストとworkerへ猶予を与えてから終了します。キャンセル後も不変条件が守られる単位にします。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結果の種類と副作用を確認します。

順序	観察対象	初心者が確認すること
1	処理期限	処理期限 — 入力と前提を確認し、境界を曖昧にしない。
2	キャンセル伝播	キャンセル伝播 — 値がどの順で変換されるかを追跡する。
3	新規受付停止	新規受付停止 — 失敗を再現し、型またはログから原因を特定する。
4	終了猶予	終了猶予 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

timeout・cancellation・graceful shutdownとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 06 · MINIMUM CODE

まず動く最小コード

timeout・cancellation・graceful shutdownだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
use std::time::Duration;
async fn slow()->u8{tokio::time::sleep(Duration::from_secs(2)).await;1}
#[tokio::main]async fn main(){match
tokio::time::timeout(Duration::from_millis(50),slow()).await{Ok(v)=>println!("{v}"),Err(_)=>println!
("timeout")}}
```

●●● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順で進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 06 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>use std::time::Duration;</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>async fn slow()- > u8{tokio::time::sleep(Duration::from_secs(2)).await;1}</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>#[tokio::main]async fn main(){match tokio::time::timeout(Duration::from_mi</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。

実行時に起きる順序

- ✓ 入力が処理期限の条件を満たすか確認する。
- ✓ キャンセル伝播を実行し、途中の型と状態を追う。
- ✓ 新規受付停止で失敗を成功経路から分離する。
- ✓ 終了猶予により、出力と副作用を検証する。

型を声に出す timeout・cancellation・graceful shutdownに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 06 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

timeout

見える結果
 端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと
 型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 06 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。


```
#[tokio::main]async fn main(){let(tx,mut rx)=tokio::sync::mpsc::channel(8);let w=tokio::spawn(async
move{while let Some(job)=rx.recv().await{println!("finish
{job}");}});tx.send(1).await.unwrap();drop(tx);w.await.unwrap();println!("shutdown complete");}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

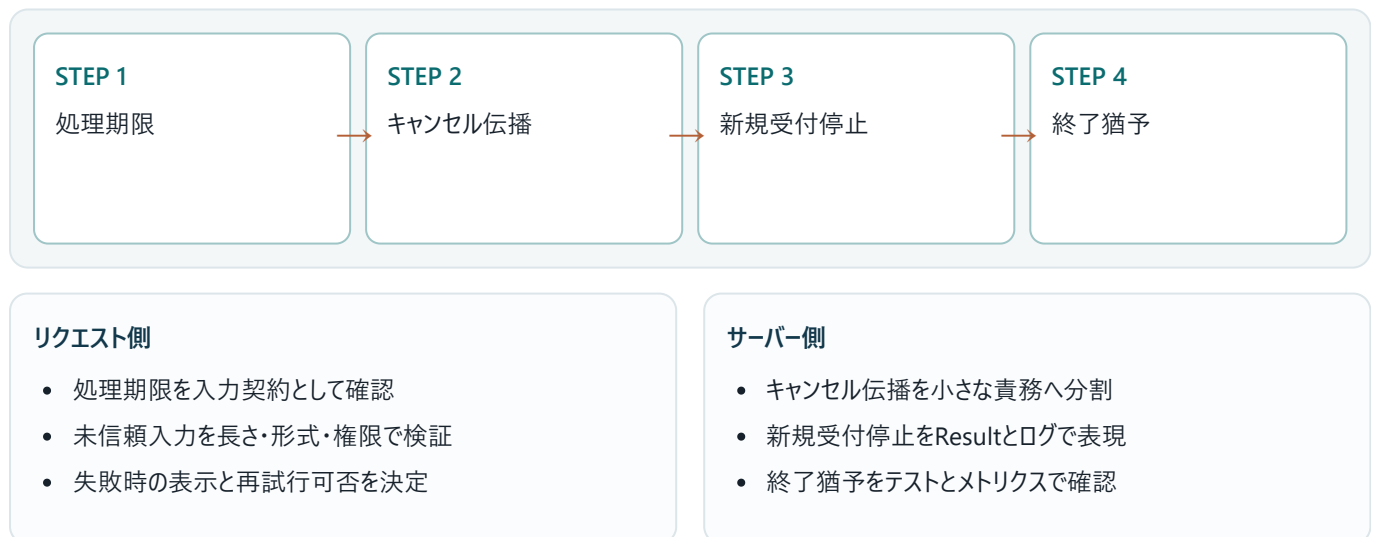
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 06 · BACKEND APPLICATION

バックエンドのどこで使うか

HTTP client、DB query、queue処理へdeadlineを伝え、途中のDB更新はtransactionで原子的にします。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 06・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	処理期限に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	キャンセル伝播の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	新規受付停止または終了猶予の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 06・DEBUGGING

timeout・cancellation・graceful shutdownを再現して切り分ける

timeout・cancellation・graceful shutdownの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 処理期限
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: キャンセル伝播
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 新規受付停止
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 終了猶予

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	処理期限	処理期限 — 入力と前提を確認し、境界を曖昧にしない。
2	キャンセル伝播	キャンセル伝播 — 値がどの順で変換されるかを追跡する。
3	新規受付停止	新規受付停止 — 失敗を再現し、型またはログから原因を特定する。
4	終了猶予	終了猶予 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 期限と終了経路を正常系と同じレベルで設計し、途中状態はtransactionで守ります。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 06 ・ PRACTICE

手を動かす演習

shutdown signal受信後にworkerへ送信側をdropし、キューを処理し終えて終了する例を作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 06 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[tokio::main]async fn main(){let(tx,mut rx)=tokio::sync::mpsc::channel(8);let w=tokio::spawn(async
move{while let Some(job)=rx.recv().await{println!("finish
{job}");}});tx.send(1).await.unwrap();drop(tx);w.await.unwrap();println!("shutdown complete");}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 期限と終了経路を正常系と同じレベルで設計し、途中状態はtransactionで守ります。
- ✓ timeout・cancellation・graceful shutdownとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 07

blocking処理・spawn_blocking・CPU負荷

UNIT 07 · CONCEPT

blocking処理・spawn_blocking・CPU負荷

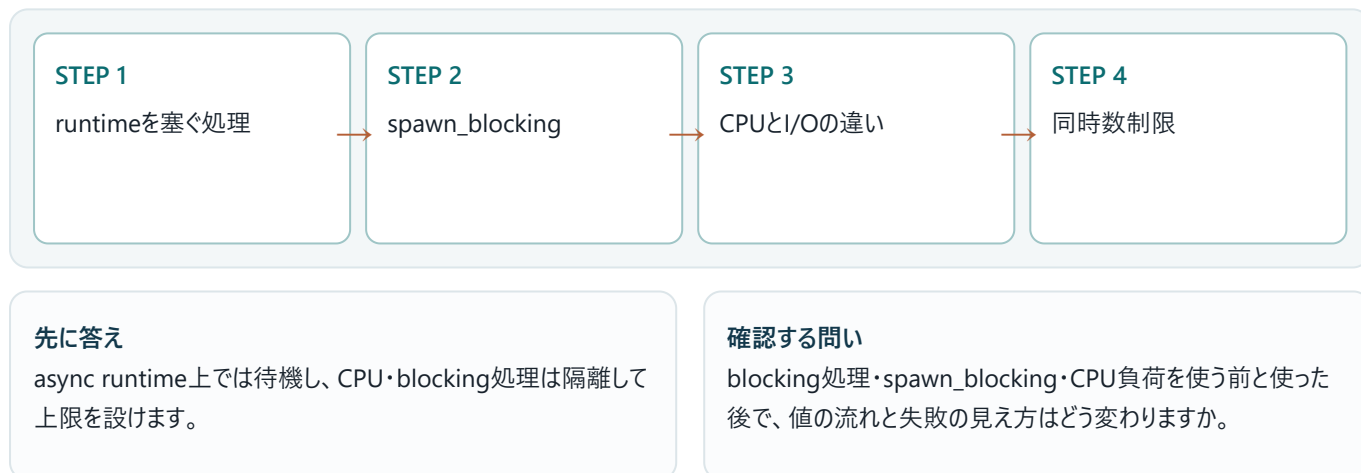
標準ファイルI/O、Argon2、圧縮など長い同期・CPU処理をruntime worker上で行うと他taskが進みません。spawn_blockingへ隔離し、同時数と入力サイズを制限します。

この単元の到達点

- runtimeを塞ぐ処理
- spawn_blocking
- CPUとI/Oの違い
- 同時数制限

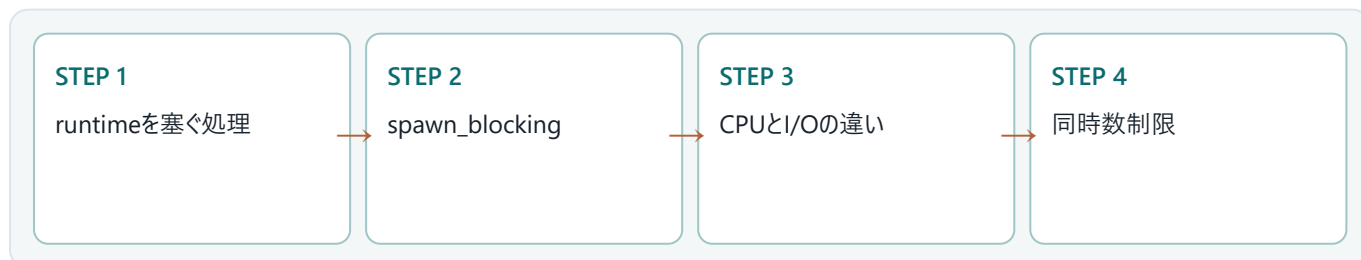
なぜバックエンドで必要か

Argon2は意図的に重いためspawn_blockingとSemaphoreを組み合わせ、攻撃者が同時ログインでCPUを使い切らないようにします。



UNIT 07 · MENTAL MODEL

blocking処理・spawn_blocking・CPU負荷を図でつかむ



標準ファイルI/O、Argon2、圧縮など長い同期・CPU処理をruntime worker上で行うと他taskが進みません。spawn_blockingへ隔離し、同時数と入力サイズを制限します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	runtimeを塞ぐ処理	runtimeを塞ぐ処理 — 入力と前提を確認し、境界を曖昧にしない。
2	spawn_blocking	spawn_blocking — 値がどの順で変換されるかを追跡する。
3	CPUとI/Oの違い	CPUとI/Oの違い — 失敗を再現し、型またはログから原因を特定する。
4	同時数制限	同時数制限 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

blocking処理・spawn_blocking・CPU負荷とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 07・MINIMUM CODE

まず動く最小コード

blocking処理・spawn_blocking・CPU負荷だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[tokio::main]async fn main(){
    let input="password".to_string();
    let len=tokio::task::spawn_blocking(move||
{std::thread::sleep(std::time::Duration::from_millis(20));input.len()}).await.unwrap();
    println!("len={len}");
}
```

●●● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 07・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[tokio::main]async fn main(){</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let input="password".to_string();</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let len=tokio::task::spawn_blocking(move {std::thread::sleep(std::time::D</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>println!("{}",len);</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力 runtime を塞ぐ処理の条件を満たすか確認する。
- ✓ `spawn_blocking` を実行し、途中の型と状態を追う。
- ✓ CPU と I/O の違いで失敗を成功経路から分離する。
- ✓ 同時数制限により、出力と副作用を検証する。

型を声に出す `blocking` 処理・`spawn_blocking`・CPU 負荷に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 07 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

`len=8`

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTP ステータス、標準エラー、ログ、DB の行数など、画面に現れない観測点も合わせて確認します。

UNIT 07 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
use std::sync::Arc; use tokio::sync::Semaphore;
async fn cpu_job(limit: Arc<Semaphore>, n: u64) -> u64 { let
    _p = limit.acquire_owned().await.unwrap(); tokio::task::spawn_blocking(move || (0..n).sum()).await.unwrap() }
#[tokio::main] async fn main() { println!("{}", cpu_job(Arc::new(Semaphore::new(2)), 100).await); }
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さず Result で返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

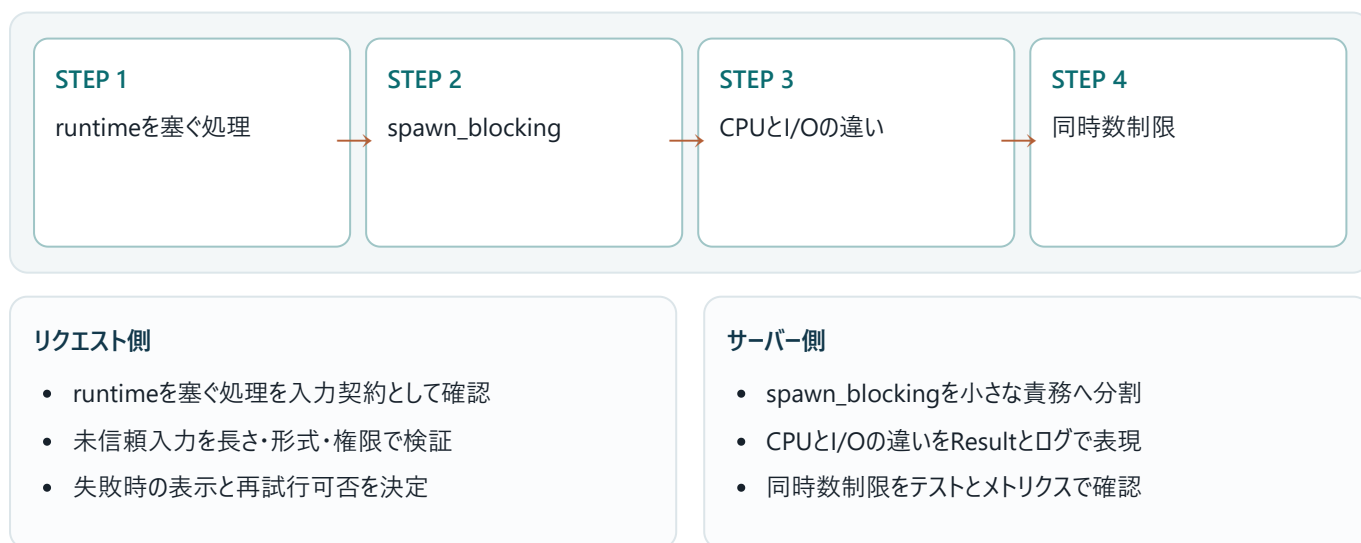
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 07 · BACKEND APPLICATION

バックエンドのどこで使うか

Argon2は意図的に重いためspawn_blockingとSemaphoreを組み合わせ、攻撃者が同時ログインでCPUを使い切らないようにします。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。

- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 07 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	runtimeを塞ぐ処理に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	spawn_blockingの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	CPUとI/Oの違いまたは同時数制限の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 07・DEBUGGING

blocking処理・spawn_blocking・CPU負荷を再現して切り分ける

blocking処理・spawn_blocking・CPU負荷の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: runtimeを塞ぐ処理
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: spawn_blocking
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: CPUとI/Oの違い
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 同時数制限

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	runtimeを塞ぐ処理	runtimeを塞ぐ処理 — 入力と前提を確認し、境界を曖昧にしない。
2	spawn_blocking	spawn_blocking — 値がどの順で変換されるかを追跡する。
3	CPUとI/Oの違い	CPUとI/Oの違い — 失敗を再現し、型またはログから原因を特定する。
4	同時数制限	同時数制限 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 async runtime上では待機し、CPU・blocking処理は隔離して上限を設けます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 07 · PRACTICE

手を動かす演習

CPU重処理をspawn_blockingへ移し、Semaphoreで同時2件までに制限してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 07 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
use std::sync::Arc; use tokio::sync::Semaphore;
async fn cpu_job(limit: Arc<Semaphore>, n: u64) -> u64 { let
    _p = limit.acquire_owned().await.unwrap(); tokio::task::spawn_blocking(move || (0..=n).sum()).await.unwrap() }
#[tokio::main] async fn main() { println!("{}", cpu_job(Arc::new(Semaphore::new(2)), 100).await); }
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ async runtime上では待機し、CPU・blocking処理は隔離して上限を設けます。
- ✓ blocking処理・spawn_blocking・CPU負荷とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 08

HTTP client・retry・idempotency

UNIT 08 ・ CONCEPT

HTTP client・retry・idempotency

外部API呼び出しはDNS、接続、TLS、応答、JSON解析で失敗します。全エラーを再試行せず、timeoutや一時的5xxだけを指数backoffとjitterで再試行し、操作の冪等性を確認します。

この単元の到達点

- 段階別timeout
- 再試行可能性
- 指数backoff
- 冪等性キー

なぜバックエンドで必要か

GETは通常再試行しやすい一方、決済やメール送信POSTは重複を防ぐidempotency keyやDB記録が必要です。



UNIT 08 ・ MENTAL MODEL

HTTP client・retry・idempotencyを図でつかむ



外部API呼び出しはDNS、接続、TLS、応答、JSON解析で失敗します。全エラーを再試行せず、timeoutや一時的5xxだけを指数backoffとjitterで再試行し、操作の冪等性を確認します。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	段階別timeout	段階別timeout — 入力と前提を確認し、境界を曖昧にしない。
2	再試行可能性	再試行可能性 — 値がどの順で変換されるかを追跡する。
3	指数backoff	指数backoff — 失敗を再現し、型またはログから原因を特定する。
4	冪等性キー	冪等性キー — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

HTTP client・retry・idempotencyとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 08・MINIMUM CODE まず動く最小コード

HTTP client・retry・idempotencyだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fn retry_delay(attempt:u32)->std::time::Duration{
    let millis=100_u64.saturating_mul(2_u64.saturating_pow(attempt.min(5)));
    std::time::Duration::from_millis(millis)
}
fn main(){for n in 0..4{println!("{:?}",retry_delay(n));}}
```

●●● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 08・LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fn retry_delay(attempt:u32)->std::time::Duration{</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>let millis=100_u64.saturating_mul(2_u64.saturating_pow(attempt.min(5)));</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>std::time::Duration::from_millis(millis)</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>}</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn main(){for n in 0..4{println!("{}",retry_delay(n));}}</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。

実行時に起きる順序

- ✓ 入力段階別timeoutの条件を満たすか確認する。
- ✓ 再試行可能性を実行し、途中の型と状態を追う。
- ✓ 指数backoffで失敗を成功経路から分離する。
- ✓ 冪等性キーにより、出力と副作用を検証する。

型を声に出す HTTP client・retry・idempotencyに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 08 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
100ms
200ms
400ms
800ms
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 08 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
fn retryable(status:u16)->bool{status==429|| (500..=599).contains(&status)}
fn attempts(statuses:&[u16])->usize{statuses.iter().take(3).take_while(|&s|retryable(s)).count()}
fn main(){assert!(retryable(503));assert!(!retryable(400));assert_eq!(attempts(&[503,503,200]),2);}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 08 · BACKEND APPLICATION

バックエンドのどこで使うか

GETは通常再試行しやすい一方、決済やメール送信POSTは重複を防ぐidempotency keyやDB記録が必要です。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。

- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 08 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	段階別timeoutに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	再試行可能性の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	指数backoffまたは冪等性キーの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 08・DEBUGGING

HTTP client・retry・idempotencyを再現して切り分ける

HTTP client・retry・idempotencyの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 段階別timeout
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 再試行可能性
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 指数backoff
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 冪等性キー

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	段階別timeout	段階別timeout — 入力と前提を確認し、境界を曖昧にしない。
2	再試行可能性	再試行可能性 — 値がどの順で変換されるかを追跡する。
3	指数backoff	指数backoff — 失敗を再現し、型またはログから原因を特定する。
4	冪等性キー	冪等性キー — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 再試行は一時失敗・上限・backoff・冪等性を一組で設計します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 08 ・ PRACTICE

手を動かす演習

429と503だけ最大3回再試行し、400は即時エラーにする判定関数を作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 08 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
fn retryable(status:u16)->bool{status==429|| (500..=599).contains(&status)}
fn attempts(statuses:&[u16])->usize{statuses.iter().take(3).take_while(|&s|retryable(s)).count()}
fn main(){assert!(retryable(503));assert(!retryable(400));assert_eq!(attempts(&[503,503,200]),2);}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 再試行は一時失敗・上限・backoff・冪等性を一組で設計します。
- ✓ HTTP client・retry・idempotencyとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 09

tracing・構造化ログ・メトリクス

UNIT 09・CONCEPT

tracing・構造化ログ・メトリクス

printlnは文字列ですが、tracingはspanとfieldでrequest_id、user_id、operation、latencyを構造化します。ログlevelと機密除外を決め、メトリクス・traceと相関できるようにします。

この単元の到達点

- eventとspan
- 構造化field
- request_id
- 機密情報の除外

なぜバックエンドで必要か

各リクエスト、DB query、外部APIを同じrequest_idで結び、エラー率・p95 latency・queue長を監視します。tokenやpasswordはfieldへ入れません。



先に答え

ログは検索できるfieldで残し、request_idと所要時間で処理を端から端まで追います。

確認する問い

tracing・構造化ログ・メトリクスを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 09・MENTAL MODEL

tracing・構造化ログ・メトリクスを図でつかむ



printlnは文字列ですが、tracingはspanとfieldでrequest_id、user_id、operation、latencyを構造化します。ログlevelと機密除外を決め、メトリクス・traceと相関できるようにします。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型

と副作用を確認します。

順序	観察対象	初心者が確認すること
1	eventとspan	eventとspan — 入力と前提を確認し、境界を曖昧にしない。
2	構造化field	構造化field — 値がどの順で変換されるかを追跡する。
3	request_id	request_id — 失敗を再現し、型またはログから原因を特定する。
4	機密情報の除外	機密情報の除外 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

tracing・構造化ログ・メトリクスとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 09・MINIMUM CODE

まず動く最小コード

tracing・構造化ログ・メトリクスだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
use tracing::{info,info_span,Instrument};
#[tokio::main]async fn main(){tracing_subscriber::fmt::init();let span=info_span!("request",request_id="req-7",method="GET",path="/notes");async{info!(rows=3,"loaded notes");}.instrument(span).await;}
```

●●● 実行コマンド

```
RUST_LOG=info cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 09・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>use tracing::{info,info_span,Instrument};</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>#[tokio::main]async fn main() {tracing_subscriber::fmt::init();let span=inf</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
tracing・構造化ログ・メトリクス	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がeventとspanの条件を満たすか確認する。
- ✓ 構造化fieldを実行し、途中の型と状態を追う。
- ✓ request_idで失敗を成功経路から分離する。
- ✓ 機密情報の除外により、出力と副作用を検証する。

型を声に出す tracing・構造化ログ・メトリクスに関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 09・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
INFO request{request_id=req-7 method=GET path=/notes}: loaded notes rows=3
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 09・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。


```
use std::time::Instant; use tracing::info;
fn run(){let start=Instant::now();let result="ok";info!(
operation="list_notes",result,elapsed_ms=start.elapsed().as_millis(),"completed");}
fn main(){tracing_subscriber::fmt::init();run();}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

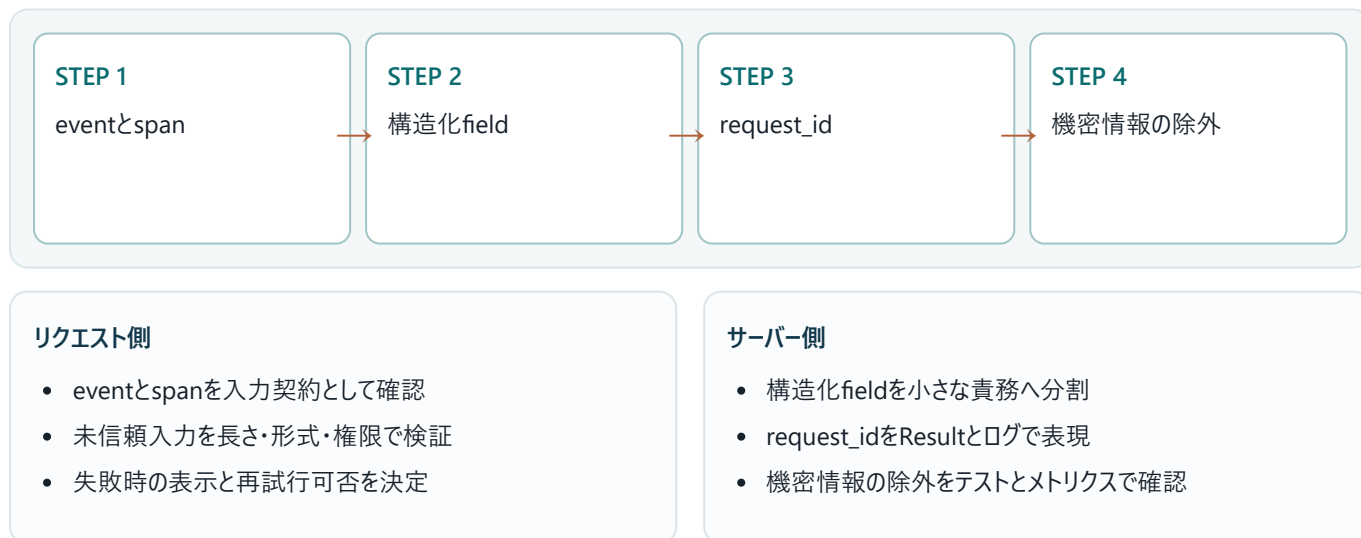
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 09 ・ BACKEND APPLICATION

バックエンドのどこで使うか

各リクエスト、DB query、外部APIを同じrequest_idで結び、エラー率・p95 latency・queue長を監視します。tokenやpasswordはfieldへ入れません。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 09・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	eventとspanに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	構造化fieldの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	request_idまたは機密情報の除外の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 09・DEBUGGING

tracing・構造化ログ・メトリクスを再現して切り分ける

tracing・構造化ログ・メトリクスの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: eventとspan
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 構造化field
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: request_id
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 機密情報の除外

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	eventとspan	eventとspan — 入力と前提を確認し、境界を曖昧にしない。
2	構造化field	構造化field — 値がどの順で変換されるかを追跡する。
3	request_id	request_id — 失敗を再現し、型またはログから原因を特定する。
4	機密情報の除外	機密情報の除外 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 ログは検索できるfieldで残し、request_idと所要時間で処理を端から端まで追います。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 09 ・ PRACTICE

手を動かす演習

処理時間をInstantで測り、operation、result、elapsed_msを構造化して記録してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 09・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
use std::time::Instant; use tracing::info;
fn run(){let start=Instant::now();let result="ok";info!(
operation="list_notes",result,elapsed_ms=start.elapsed().as_millis(),"completed");}
fn main(){tracing_subscriber::fmt::init();run();}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ ログは検索できるfieldで残し、request_idと所要時間で処理を端から端まで追います。
- ✓ tracing・構造化ログ・メトリクスとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 10

background job・設定・運用境界

UNIT 10・CONCEPT

background job・設定・運用境界

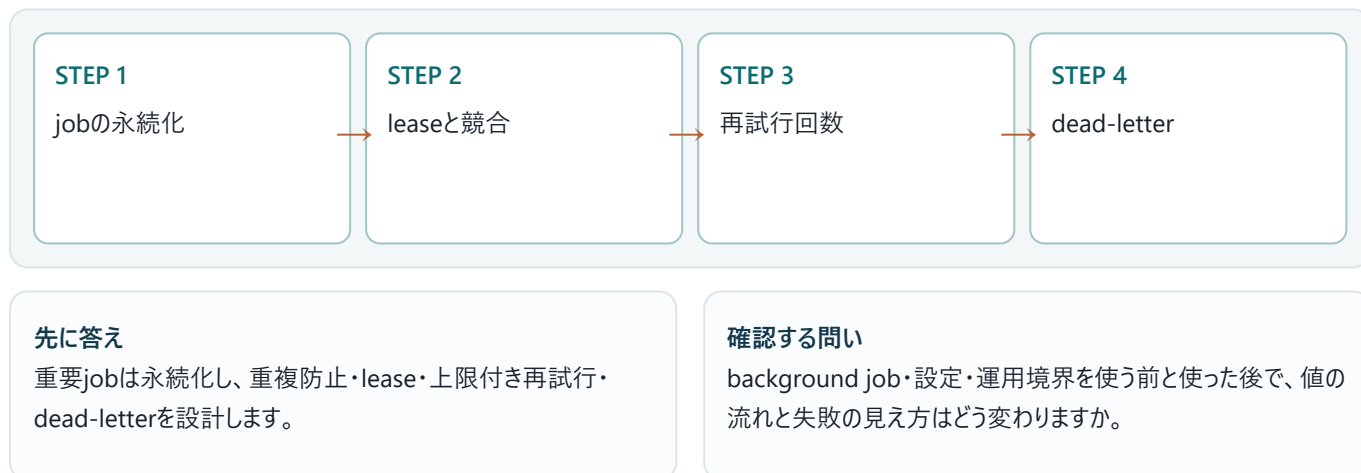
バックグラウンド処理には投入、予約、取得、lease、成功、再試行、dead-letterの状態があります。メモリtaskでは再起動に弱いため、重要処理はDBへ永続化し複数workerでも一度だけ実行します。

この単元の到達点

- jobの永続化
- leaseと競合
- 再試行回数
- dead-letter

なぜバックエンドで必要か

メール、監査、集計をHTTP応答から分離します。jobの一意キー、最大試行、次回時刻、最後のエラー分類をDBへ持たせます。



UNIT 10・MENTAL MODEL

background job・設定・運用境界を図でつかむ



バックグラウンド処理には投入、予約、取得、lease、成功、再試行、dead-letterの状態があります。メモリtaskでは再起動に弱いため、重要処理はDBへ永続化し複数workerでも一度だけ実行します。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	jobの永続化	jobの永続化 — 入力と前提を確認し、境界を曖昧にしない。
2	leaseと競合	leaseと競合 — 値がどの順で変換されるかを追跡する。
3	再試行回数	再試行回数 — 失敗を再現し、型またはログから原因を特定する。
4	dead-letter	dead-letter — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

background job・設定・運用境界とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 10・MINIMUM CODE

まず動く最小コード

background job・設定・運用境界だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
#[derive(Debug)]enum JobState{Pending,Running{worker:String},Succeeded,Failed{attempts:u8}}
fn retry(state:JobState)->JobState{match state{JobState::Failed{attempts} if
attempts<3=>JobState::Pending,other=>other}}
fn main(){println!("{:?}",retry(JobState::Failed{attempts:1}));}
```

● ● ● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 10・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>#[derive(Debug)]enum JobState{Pending,Running{worker:String},Succeeded,Fai</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>fn retry(state:JobState)->JobState{match state{JobState::Failed{attempts}</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
<code>fn main(){println!("{}",retry(JobState::Failed{attempts:1}});}</code>	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。

実行時に起きる順序

- ✓ 入力がjobの永続化の条件を満たすか確認する。
- ✓ leaseと競合を実行し、途中の型と状態を追う。
- ✓ 再試行回数で失敗を成功経路から分離する。
- ✓ dead-letterにより、出力と副作用を検証する。

型を声に出す background job・設定・運用境界に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 10 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

Pending

見える結果
 端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと
 型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 10 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。


```
#[derive(Debug,PartialEq)]enum State{Pending,DeadLetter}
fn after_failure(attempts:u8)->State{if attempts>=3{State::DeadLetter}else{State::Pending}}
fn main(){assert_eq!(after_failure(2),State::Pending);assert_eq!(after_failure(3),State::DeadLetter);}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

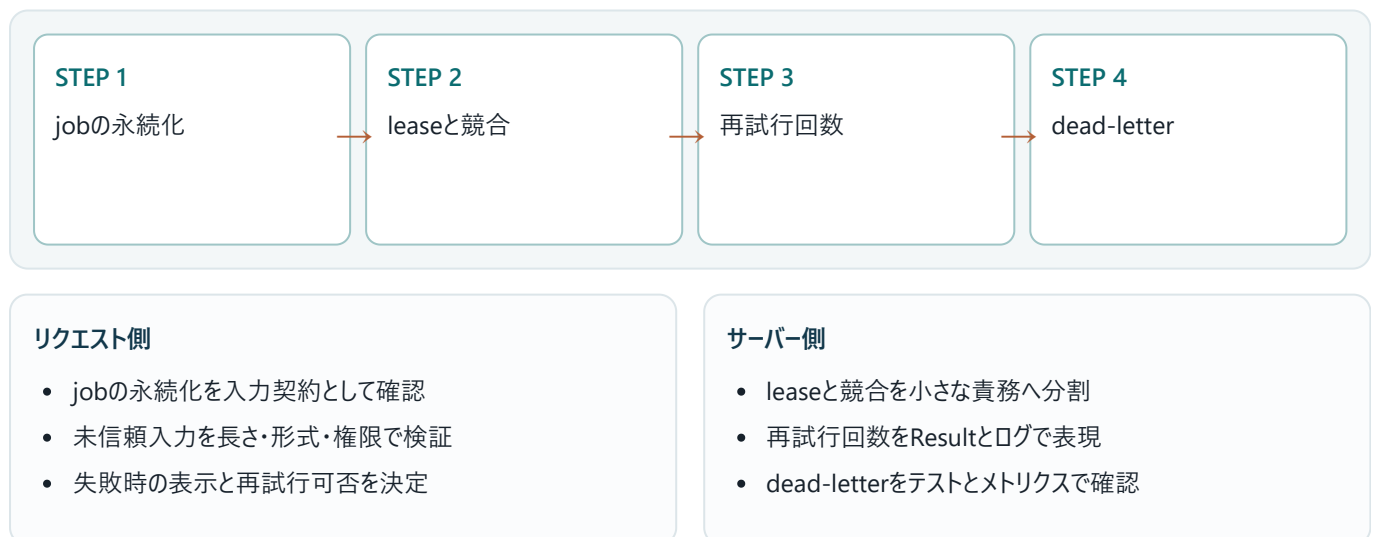
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 10 ・ BACKEND APPLICATION

バックエンドのどこで使うか

メール、監査、集計をHTTP応答から分離します。jobの一意キー、最大試行、次回時刻、最後のエラー分類をDBへ持たせます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 10・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	jobの永続化に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	leaseと競合の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	再試行回数またはdead-letterの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 10・DEBUGGING

background job・設定・運用境界を再現して切り分ける

background job・設定・運用境界の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: jobの永続化
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: leaseと競合
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 再試行回数
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: dead-letter

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	jobの永続化	jobの永続化 — 入力と前提を確認し、境界を曖昧にしない。
2	leaseと競合	leaseと競合 — 値がどの順で変換されるかを追跡する。
3	再試行回数	再試行回数 — 失敗を再現し、型またはログから原因を特定する。
4	dead-letter	dead-letter — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 重要jobは永続化し、重複防止・lease・上限付き再試行・dead-letterを設計します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 10 ・ PRACTICE

手を動かす演習

Job構造体にidempotency_keyとattemptsを持たせ、3回失敗したらDeadLetterへ遷移してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 10 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[derive(Debug, PartialEq)]enum State{Pending, DeadLetter}
fn after_failure(attempts:u8)->State{if attempts>=3{State::DeadLetter}else{State::Pending}}
fn main(){assert_eq!(after_failure(2),State::Pending);assert_eq!(after_failure(3),State::DeadLetter);}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 重要jobは永続化し、重複防止・lease・上限付き再試行・dead-letterを設計します。
- ✓ background job・設定・運用境界とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

CAPSTONE

期限、同時数、停止経路、構造化ログを備えた非同期 worker を作る

10単元を一つの成果物へ統合します。動く結果だけでなく、途中のエラー、設計判断、確認コマンドも提出物です。

順	使う単元	統合する判断
01	async・Future・awaitの実体	asyncは待機を表すFutureを作り、await地点でexecutorへ制御を返します。
02	Tokio runtime・scheduler・spawn	spawnしたタスクには所有者と結果回収経路を用意します。
03	JoinSet・select!・構造化された並行処理	並行taskを放置せず、親が完了・失敗・キャンセルを回収します。
04	channel・backpressure・所有権移動	bounded channelで処理能力を明示し、重要jobは永続化と再試行を設計します。
05	Mutex・RwLock・Semaphoreとawait	ロックは状態、Semaphoreは同時数を守り、awaitをまたぐguardを慎重に扱います。
06	timeout・cancellation・graceful shutdown	期限と終了経路を正常系と同じレベルで設計し、途中状態はtransactionで守ります。
07	blocking処理・spawn_blocking・CPU負荷	async runtime上では待機し、CPU・blocking処理は隔離して上限を設けます。
08	HTTP client・retry・idempotency	再試行は一時失敗・上限・backoff・冪等性を一組で設計します。
09	tracing・構造化ログ・メトリクス	ログは検索できるfieldで残し、request_idと所要時間で処理を端から端まで追います。
10	background job・設定・運用境界	重要jobは永続化し、重複防止・lease・上限付き再試行・dead-letterを設計します。

完了条件

- ✓ bounded queueとSemaphoreがある
- ✓ 全I/Oにtimeoutがある
- ✓ Ctrl+Cで安全にdrainする
- ✓ 失敗jobが上限付きで再試行される

REVIEW

巻末確認問題

用語だけでなく、「小さな例」「バックエンドでの場所」「失敗時の挙動」を含めて教えてください。

1. 01. `async`・`Future`・`await`の実体を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
2. 02. `Tokio runtime`・`scheduler`・`spawn`を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
3. 03. `JoinSet`・`select!`・構造化された並行処理を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
4. 04. `channel`・`backpressure`・所有権移動を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
5. 05. `Mutex`・`RwLock`・`Semaphore`と`await`を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
6. 06. `timeout`・`cancellation`・`graceful shutdown`を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
7. 07. `blocking`処理・`spawn_blocking`・CPU負荷を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
8. 08. `HTTP client`・`retry`・`idempotency`を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
9. 09. `tracing`・構造化ログ・メトリクスを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
10. 10. `background job`・設定・運用境界を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

答える順序 まず資料を閉じて説明し、次に最小コードを書き、最後に該当単元へ戻って不足を補います。正解の暗記ではなく、根拠を再現できることが目標です。

REVIEW ANSWERS

確認問題・解答の骨格

次の要点は模範解答の丸暗記用ではありません。自分の説明に「定義・小さな例・バックエンドでの場所」が含まれているかを照合します。

01. async・Future・awaitの実体

async・Future・awaitの実体とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: asyncは待機を表すFutureを作り、await地点でexecutorへ制御を返します。

02. Tokio runtime・scheduler・spawn

Tokio runtime・scheduler・spawnとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: spawnしたタスクには所有者と結果回収経路を用意します。

03. JoinSet・select!・構造化された並行処理

JoinSet・select!・構造化された並行処理とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 並行taskを放置せず、親が完了・失敗・キャンセルを回収します。

04. channel・backpressure・所有権移動

channel・backpressure・所有権移動とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: bounded channelで処理能力を明示し、重要jobは永続化と再試行を設計します。

05. Mutex・RwLock・Semaphoreとawait

Mutex・RwLock・Semaphoreとawaitとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: ロックは状態、Semaphoreは同時数を守り、awaitをまたぐguardを慎重に扱います。

06. timeout・cancellation・graceful shutdown

timeout・cancellation・graceful shutdownとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 期限と終了経路を正常系と同じレベルで設計し、途中状態はtransactionで守ります。

07. blocking処理・spawn_blocking・CPU負荷

blocking処理・spawn_blocking・CPU負荷とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: async runtime上では待機し、CPU・blocking処理は隔離して上限を設けます。

08. HTTP client・retry・idempotency

HTTP client・retry・idempotencyとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 再試行は一時失敗・上限・backoff・冪等性を一組で設計します。

09. tracing・構造化ログ・メトリクス

tracing・構造化ログ・メトリクスとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: ログは検索できるfieldで残し、request_idと所要時間で処理を端から端まで追います。

10. background job・設定・運用境界

background job・設定・運用境界とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 重要jobは永続化し、重複防止・lease・上限付き再試行・dead-letterを設計します。

GLOSSARY

重要用語と実務での位置

用語	一文定義	バックエンドでの位置
async・Future・awaitの実体	asyncは待機を表すFutureを作り、await地点でexecutorへ制御を返します。	DB、HTTP、タイマーの待機をasyncで表し、少数スレッドで多数接続を扱います。CPU重処理が速くなる仕組みではありません。
Tokio runtime・scheduler・spawn	spawnしたタスクには所有者と結果回収経路を用意します。	リクエスト処理から無制限にspawnすると障害時にタスクが増え続けます。所有者、終了条件、同時数、失敗回収を設計します。
JoinSet・select!・構造化された並行処理	並行taskを放置せず、親が完了・失敗・キャンセルを回収します。	クライアント切断、サーバー停止、タイムアウトのいずれかが先に起きたら処理を止め、DB更新の途中状態をトランザクションで守ります。
channel・backpressure・所有権移動	bounded channelで処理能力を明示し、重要jobは永続化と再試行を設計します。	メールや監査イベントをリクエストからworkerへ渡します。メモリchannelだけでは再起動で消えるため、重要jobはDB queueへ永続化します。
Mutex・RwLock・Semaphoreとawait	ロックは状態、Semaphoreは同時数を守り、awaitをまたぐguardを慎重に扱います。	外部APIや重いDB処理の同時数をSemaphoreで制限し、過負荷時の待機上限と拒否方針を決めます。
timeout・cancellation・graceful shutdown	期限と終了経路を正常系と同じレベルで設計し、途中状態はtransactionで守ります。	HTTP client、DB query、queue処理へdeadlineを伝え、途中のDB更新はtransactionで原子的にします。
blocking処理・spawn_blocking・CPU負荷	async runtime上では待機し、CPU・blocking処理は隔離して上限を設けます。	Argon2は意図的に重いためspawn_blockingとSemaphoreを組み合わせ、攻撃者が同時ログインでCPUを使い切らないようにします。
HTTP client・retry・idempotency	再試行は一時失敗・上限・backoff・冪等性を一組で設計します。	GETは通常再試行しやすい一方、決済やメール送信POSTは重複を防ぐidempotency keyやDB記録が必要です。
tracing・構造化ログ・メトリクス	ログは検索できるfieldで残し、request_idと所要時間で処理を端から端まで追います。	各リクエスト、DB query、外部APIを同じrequest_idで結び、エラー率・p95 latency・queue長を監視します。tokenやpasswordはfieldへ入れません。
background job・設定・運用境界	重要jobは永続化し、重複防止・lease・上限付き再試行・dead-letterを設計します。	メール、監査、集計をHTTP応答から分離します。jobの一意キー、最大試行、次回時刻、最後のエラー分類をDBへ持たせます。

PRIMARY SOURCES

公式資料と更新確認

仕様は更新されます。教材で意図を理解した後、実装時点の一次資料でAPI、security note、breaking changeを確認してください。

1. Rust Book

<https://doc.rust-lang.org/stable/book/>
Rust 2024 Edition を前提にした公式入門書

2. Rust Reference

<https://doc.rust-lang.org/reference/>
言語仕様を確認する一次資料

3. Standard Library

<https://doc.rust-lang.org/std/>
標準ライブラリAPI

4. Cargo Book

<https://doc.rust-lang.org/cargo/>
ビルド、依存関係、workspace、features

5. Rustdoc Book

<https://doc.rust-lang.org/rustdoc/>
APIドキュメントの書き方

6. Clippy

<https://doc.rust-lang.org/clippy/>
Rust公式Lint

7. Async Book

<https://rust-lang.github.io/async-book/>
非同期Rustの公式解説

8. Tokio Tutorial

<https://tokio.rs/tokio/tutorial>
Tokio公式チュートリアル

9. Topcoat

<https://github.com/tokio-rs/topcoat>
Topcoat公式リポジトリ。early-stageの注意を含む

10. Topcoat 0.5.0 API

<https://docs.rs/topcoat/0.5.0/topcoat/>
本教材で固定するAPI

11. Turso Rust SDK

<https://docs.turso.tech/sdk/rust/quickstart>
libSQL Rust SDK公式手順

12. Better Auth Database

<https://better-auth.com/docs/concepts/database>
Better AuthのDB構成

13. Fly.io Deploy

<https://fly.io/docs/launch/deploy/>
fly launch / fly deploy公式手順

14. Fly.io Health Checks

<https://fly.io/docs/reference/health-checks/>
公開後のヘルスチェック

15. OWASP Cheat Sheets

<https://cheatsheetseries.owasp.org/>
Webセキュリティの実務チェック

採用判断 本教材はRust 2024 Edition、Topcoat 0.5.0固定、Turso/libSQL、Topcoat Session + Argon2id、Fly.ioを主経路にします。Better AuthはTypeScript sidecarが適切な要件で比較します。